MATH 71.1
Ch.2 SALT 🧂
(Stopped from Appearing in the Long Test)

October 22, 2025

Access the contest website at the following url:

https://tinyurl.com/SALT-Ch2-2526-1

Hidden Test Cases. Your solution will be checked by running it against one or more (usually several) hidden test cases. **You will not have access to these cases**, but a correct solution is expected to handle them correctly.

Not Sorted by Difficulty. The problem set is **not** sorted by difficulty! Each team is encouraged to have at least read all the problems by the end of the contest.

Submit Code Only. Only include **one** file when submitting: the source code (.py) and nothing else.

Only Builtin Libraries. Do not use third-party libraries. Only libraries that come installed with the compilers are available on the evaluation machines.

No Weird Filenames. Only use letters, digits and underscores in your filename. Do not use spaces or other special symbols.

Good luck and enjoy the contest! 😃

Problem A
Absolute Cinema

For Bob's art class, he was tasked with making art of a mountain range—but, it must be done in a creative way, and the more creative the better, especially if it incorporates Bob's interests. Alice suggests that he could model the mountain range using mathematics.

What a nerd!

Given a sequence $a_1, a_2, a_3, \ldots, a_n$ of positive **even** integers, let

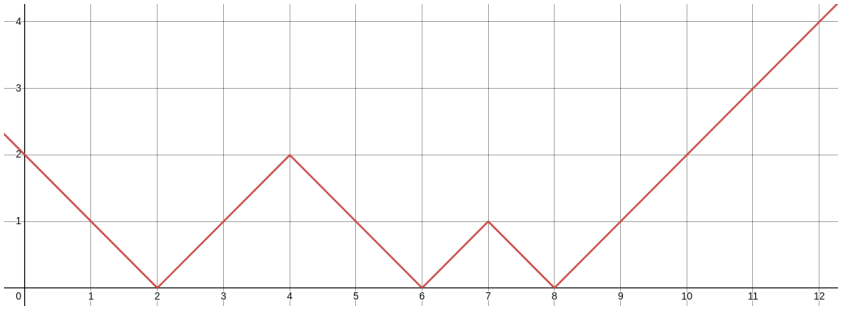
$$A(x) = \min\left(|x - a_1|,\ |x - a_2|,\ |x - a_3|, \ldots,\ |x - a_n|\right).$$

Alice says that this function $A(x)$ resembles a mountain range when its graph is plotted using a graphing calculator.

For example, if $a = [6, 2, 8, 2]$ and $r = 12$, then the graph of

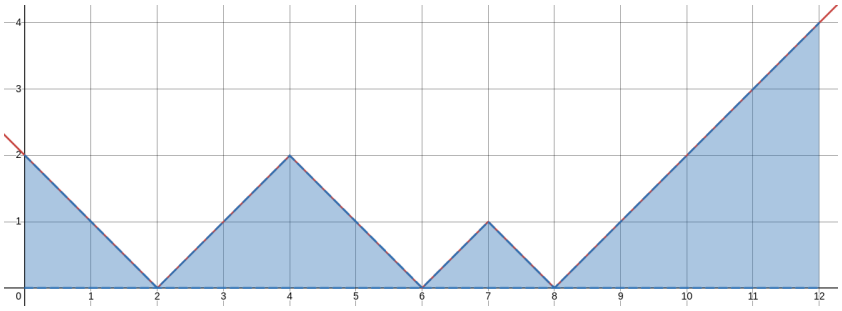
$$A(x) = \min(|x - 6|,\ |x - 2|,\ |x - 8|,\ |x - 2|)$$

would be the one illustrated below.



Bob would like to create a mountain range by defining the function $A(x)$ using his favorite integer sequence $a_1, a_2, a_3, \ldots, a_n$. His painting will span from $x = 0$ to $x = r$, and he will use blue paint to shade in the region between $A(x)$ and the x-axis.

For example, if $a = [6, 2, 8, 2]$ and $r = 12$, then the Bob would want to paint blue the highlighted region in the diagram below.



Its total area can be shown to be `15` exactly.

In fact, Bob is able to show that if the a_i are all even, and r is also even, then the area of the desired region is **guaranteed** to be an integer!

Given all these values, please determine the total *area* of this region, so that Bob knows how much paint to buy. It's *integral* that he knows this information, *definitely*!

Input Format

Input begins with a single line containing the space-separated positive integers n and r, where r is even.

The second line of input contains the n space-separated even positive integers $a_1, a_2, a_3, \ldots, a_n$.

Output Format

Output a line containing a single **integer**, the area of the desired region.

Constraints

Your program will only be tested on data that satisfies the following constraints.

Constraints
$1 \leq n \leq 100$ $2 \leq r \leq 1000$ and r is even $2 \leq a_i \leq 1000$ and a_i is even, for each i

There is no need to validate in your program that these constraints hold.

Sample I/O

Input	Output
`4 12` `6 2 8 2`	`15`

Input	Output
`1 2` `2`	`2`

Problem B
Kabayos!

2012 Philippine Triple Crown winner Hagdang Bato*, with his jockey JB Hernandez.*

Image taken from the Philippine Daily Inquirer

Cindy is addicted to a brand new horse-racing *gacha* mobile game. The game is a *raising simulator*, meaning the main gameplay loop is the process of training a character. In this game, each of your characters is a *kabayo*.

Each kabayo has n attributes called "stats", each with some positive integer value. There exists a *stat cap* of c, meaning c is the maximum value that any stat can be.

When one playthrough of the game is completed, Cindy's kabayo is assigned a numerical rating that is computed based on its stats, the higher the better.

However, the rating formula is not linear. The rating is computed as the *sum of the squares* of each of Alice's kabayo's stats. For example, with $n = 5$, a kabayo with stats $[3, 1, 4, 1, 5]$ would have a rating of

$$3^2 + 1^2 + 4^2 + 1^2 + 5^2 = 52.$$

Cindy's kabayo begins with some initial stats. She then will decide how to train her kabayo, meaning she may do the following action at most k times:

- Choose one of her kabayo's stats with a value strictly less than c;
  increase its value by $+1$.

You are given the initial stats of Cindy's kabayo, as well as c and k. If she raises her kabayo optimally, what is the maximum possible rating she can get?

Input Format

Input begins with a single line containing the space-separated integers n and c and k.

The second line of input contains n space-separated integers, the initial values of Cindy's kabayo's stats.

Output Format

Output a line containing a single integer, the maximum possible rating.

Constraints

Your program will only be tested on data that satisfies the following constraints.

Constraints
$1 \leq n \leq 100$ $1 \leq k \leq 2000$ $1 \leq c \leq 1200$ $1 \leq$ each initial stat $\leq c$

There is no need to validate in your program that these constraints hold.

Sample I/O

Input	Output
5 6 2 3 1 4 1 5	72

Input	Output
5 6 1 3 1 4 1 5	63

Input	Output
5 5 1 3 1 4 1 5	61

Input	Output
1 1200 99 1	10000

Problem C

Light Game

Alice is replaying one of her favorite childhood platformers, *Spongeboy: Skirmish for Scarborough Shoal*. In this game, Spongeboy must defend his home from a "robot" invasion. *Supposedly*, it's said that the robots are an allegory for something, but Alice isn't quite sure what... Metaphors are hard to decipher!

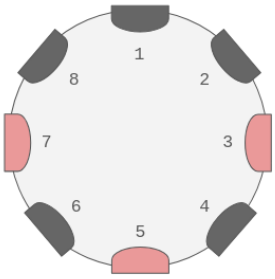
Aside from the main story, there are many puzzles which serve as side-objectives which can be completed for additional rewards. One of these is the infamous *Light Game*.

Spongeboy is dropped into a circular room, with n lights arranged in sequence along the wall. The lights are numbered 1 to n in the clockwise direction. More formally,

- for each $1 < i \leq n$, light $i-1$ is the left neighbor of light i
  (also, light n is the left neighbor of light 1);
- for each $1 \leq i < n$, light $i+1$ is the right neighbor of light i
  (also, light 1 is the right neighbor of light n).

Each light exists in one of two states: *off* or *on*.

Here is an example layout of the room with $n = 8$. Lights 3 and 5 and 7 are initially on.



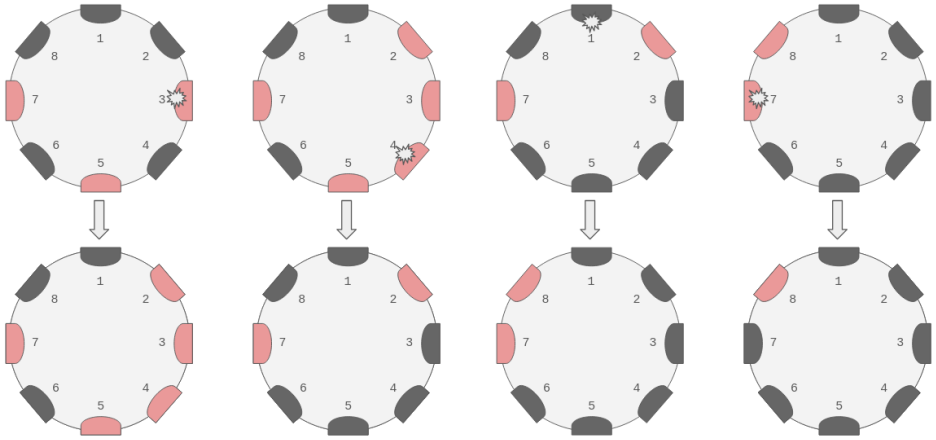
Each light is also a button that you can press. When you press a light, the states of it and/or its neighbors may change, according to some seemingly random behavior.

Alice has a keen eye though, and deduced that the lights' behavior follows these consistent (if peculiar) rules:

- If you press a light where both its neighbors are off,
  both its neighbors become on
- If you press a light where both its neighbors are on,
  its state becomes flipped *and* both its neighbors become off.
- If the pressed light has the same state as its left neighbor but not its right,
  the states of its two neighbors *both* become flipped.
- If the pressed light has the same state as its right neighbor but not its left,
  the state of the pressed light changes to become the same as its *left* neighbor.

Here, "flipping a state" means: off lights become on, and on lights become off.

Following the same example setup of $n = 8$ lights from earlier (where initially, lights 3 and 5 and 7 are on): if we press light 3, then light 4, then light 1, then light 7, we get the sequence of state changes illustrated in the diagram below.



We press light 3*, then press light* 4*, then press light* 1*, then press light* 7*.*

You are given an initial configuration of lights, as well as the target configuration that Spongeboy must reach. Does there exist *any* sequence of button presses which achieves this goal? If yes, please also construct such a solution.

Input Format

Input consists of two lines, each containin a string of n characters. The first line represents the initial configuration, and the second line represents the target configuration.

The ith character is `1` if light i is (or must be) on, and `0` if it is (or must be) off. You are reminded that the room is *circular*—despite the appearance of the input format, the first and last lights are actually next to each other.

Output Format

Output a line containing `YES` if possible, and `NO` if not.

Then, if `YES`, output a solution. First, output a line containing a non-negative integer m, the number of button presses in your construction. Then, if $m > 0$, output another line containing m space-separated integers, where each integer is from 1 to n. This encodes the instructions of which lights should be pressed, to be done in the order they are given.

Your solution must satisfy $m \leq 3 \cdot 10^5$. Given the constraints, it can be shown that if a solution exists, then one satisfying $m \leq 3 \cdot 10^5$ exists. If there are multiple possible answers, any will be accepted.

Constraints

Your program will only be tested on data that satisfies the following constraints.

Constraints
$3 \leq n \leq 100$ The input strings consist of `0` and `1` only.

There is no need to validate in your program that these constraints hold.

Sample I/O

Input	Output
00101010 00000001	YES 4 3 4 1 7

Input	Output
000 111	YES 3 3 3 3

Input	Output
01010 01010	YES 0

Problem D
Love Game

Says Luna (translated as *Ani Buwan*) is a video game under the genre of *dating simulator*. You play the role of a young man who has inherited his grandfather's old farm, tasked with both adapting to rural life and forming bonds with your new community.

People in the community have an *affection rating* for you, represented as some integer, possibly zero or negative (where negative affection corresponds to disdain). Affection ratings are modified by giving these characters gifts.

Cindy is playing the game, and wants to reach an affection rating of t or more with her favorite *husbando*. This husbando initially has an affection rating of s.

However, at this stage of the game, Cindy only has access to one kind of item, the *Square Seed*. If she gives one Square Seed to her husbando, it has the following effect on his affection rating:

- If his affection rating is currently x, then after receiving one Square Seed, it becomes $x^2 - 3$.

So for example, if $s = -4$, then after giving the husbando one Square Seed, his affection rating becomes 13. After giving him another Square Seed, his affection rating then becomes 166.

Given s and t, determine the minimum number of Square Seeds Cindy must give her husbando in order for his affection rating to become $\geq t$. If this is a fool's errand though (i.e. impossible no matter how many square seeds you give him), please tell Cindy.

Input Format

Input consists of a single line containing the two space-separated integers s and t.

Output Format

If the task is impossible even if Cindy had all the Square Seeds in the world, output the words "`MOVE ON`" (without the quotes).

Otherwise, output a single non-negative integer, the minimum number of Square Seeds that Cindy needs to give her husbando.

Constraints

Your program will only be tested on data that satisfies the following constraints.

Constraints
$-10^9 \leq s, t \leq 10^9$

There is no need to validate in your program that these constraints hold.

Sample I/O

Input	Output
-4 100	2

Input	Output
-99 -100	0

Input	Output
0 1000	4

Problem E

Mixed Messages

I have a puzzle for you! Can you decipher the hidden meaning in the following message?

4427777

This is a classic cipher. Here's a hint:

Image taken from Wikipedia

On an old cellphone keypad, letters are typed by pressing the corresponding key some number of times. Each key is responsible for multiple letters (in the order they are written on the keypad), and each tap of the key would let you *cycle* through your options of which letter gets typed in.

For example, tapping `2` once would encode `a`, tapping it twice would encode `b`, and tapping it thrice would encode `c`. One would "lock in" a choice by pausing before tapping again (usually you would have to wait for around one second between inputs).

We can use this to create a simple cipher. Replace each English letter with the corresponding digit of the key it belongs to, repeated a number of times equal to the number of times you would tap that digit to get that letter.

For example, to translate the string `has`, we note that: `h` is `44`; `a` is `2`; and `s` is `7777`. Concatenating them, we see that `has` is `4427777`.

The issue with this scheme is that by not encoding the pauses, it is posisble to have multiple words that map to the same ciphertext, making the message ambiguous.

For example, to translate the string `harp`, we note that: `h` is `44`; `a` is `2`; `r` is `777`; and `p` is `7`. Concatenating them, we see that `harp` is *also* `4427777`.

Given a string of digits (each from `2` to `9`) to be interpreted as ciphertext, determine if the message is clear (if there is exactly one plaintext string which produces that ciphertext) or ambiguous (if multiple possible plaintext strings are encoded as that ciphertext). If the message is clear, please decode it for us as well!

Input Format

Input consists of a single line containing a string of digits (each from `2` to `9`), the ciphertext.

Output Format

First, output a line containing either the string `CLEAR` or `AMBIGUOUS` according to the uniqueness of how to decode the ciphertext.

If `CLEAR`, also output another line containing the unique decrypted message (in all-lowercase letters).

Constraints

Your program will only be tested on data that satisfies the following constraints.

Constraints
The ciphertext consists of at least 1 and at most 100 digits, each from `2` to `9`.

There is no need to validate in your program that these constraints hold.

Sample I/O

Input	Output
`725262`	`CLEAR` `pajama`

Input	Output
`4427777`	`AMBIGUOUS`

Problem F
Six Seven

Alice was tasked with implementing Dijkstra's shunting yard, an algorithm for parsing arithmetic expressions, respecting correct operator precedence (i.e. MDAS). However, she submitted a meme instead, which annoyed her teacher.

- The `+` operation is implemented as **string concatenation** instead of addition, e.g. `6 + 7` evaluates to `67`
- The `*` operation is implemented as **string duplication** instead of multiplication, e.g. `6 * 7` evaluates to `6666666`
- The `-` operation is implemented as normal, except negative numbers were not implemented, so if any operation would result in one at any intermediate step (e.g. `6 - 7`) then the computer explodes.
- The `/` operation is implemented as normal, except non-integer fractions were not implemented, so if any operation would result in one at any intermediate step (e.g. `6 / 7`) then the computer explodes.
- Finally, the **only** accepted integer literals are `6` and `7`.

Otherwise, her program correctly follows the MDAS convention.

- It recursively evaluates parenthesized expressions first, then
- resolves `*` and `/` operations from left to right, then
- resolves `+` and `-` operations from left to right.

Alice's program ignores spaces between tokens.

For example, `7 - 6 + 6` equals `16`, evaluated as follows:

$$\begin{aligned} \texttt{7 - 6 + 6} &= \texttt{1 + 6} \\ &= \texttt{16}. \end{aligned}$$

For another example, `(7+7) - (6+7 - 7)/6` equals `67`, evaluated as follows:

$$\begin{aligned} \texttt{(7+7) - (6+7 - 7)/6} &= \texttt{77 - (6+7 - 7)/6}, \\ &= \texttt{77 - (67 - 7)/6}, \\ &= \texttt{77 - 60/6}, \\ &= \texttt{77 - 10} \\ &= \texttt{67}. \end{aligned}$$

For one final example, `6*6 / (6 + (6 + 6)) / 7` equals `143`, evaluated as follows:

$$\begin{aligned} \texttt{6*6 / (6 + (6 + 6)) / 7} &= \texttt{6*6 / (6 + 66) / 7} \\ &= \texttt{6*6 / 666 / 7} \\ &= \texttt{666666 / 666 / 7} \\ &= \texttt{1001 / 7} \\ &= \texttt{143}. \end{aligned}$$

Given a positive integer n, your goal is to construct any arithmetic expression (using parentheses, `6` and `7`, and the `*/+-` operations) which evaluates to n using Alice's meme program. *For completeness*, here are the other limitations of Alice's program:

- If the expression contains more than 1000 characters (including spaces), the computer blows up.
- If at any intermediate step an integer with a value greater than 2^{64} is produced, the computer blows up.
- For `+` and `*`, leading zeros are removed at each intermediate step, e.g. `0 + 0` and `0 * 5` both evaluate to `0` (just one `0`)
- Finally, `n * 0` causes the computer to blow up, regardless of the left operand.

Input Format

Input consists of a single positive integer n.

Output Format

Output a single line containing *any* valid expression which evaluates to n.

We give the following formal definition of what exactly constitutes an *expression*.

An *expression* is a sequence of one or more sub-expressions, separated by operators.
A *sub-expression* is one of the following:

- an integer literal (which must be `6` or `7`); or
- another expression, enclosed in parentheses `()`

Each operator must be one of `+` or `-` or `*` or `/`

Spaces between tokens are ignored, so add as many as you like (within the char. count).

Here are some more examples of valid expressions:

```
(7) - ((6)) + (((7)))
(6 + 7 * 6)
(6 / 6) * (7)
(((((((6)))))))
7
```

These evaluate to `17` and `6777777` and `1111111` and `6` and `7`, respectively.

Constraints

Your program will only be tested on data that satisfies the following constraints.

Constraints
$1 \leq n \leq 10^{12}$

There is no need to validate in your program that these constraints hold.

Sample I/O

Input	Output
16	7 - 6 + 6

Input	Output
67	(7+7) - (6+7 - 7)/6

Input	Output
143	6*6 / (6 + (6 + 6)) / 7

Input	Output
17	(7) - ((6)) + (((7)))

Input	Output
6777777	(6 + 7 * 6)

Input	Output
1111111	(6 / 6) * (7)

Input	Output
6	(((((((6)))))))

Input	Output
7	7

Problem G
Tetris Fanfiction

A *tetromino* is a shape formed by connecting four unit squares together edge-to-edge. If we count rotations as fundamentally the same shape (but reflections are distinct!), then there are *seven* different tetrominos, illustrated below. Each is commonly associated with an uppercase English letter whose shape it resembles.

Image taken from Wikipedia

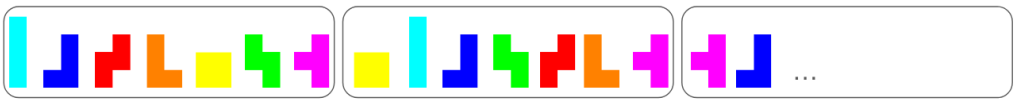
Tetris is a game in which you are randomly given tetrominos one at a time, and must place them onto to the gameboard in the order you receive them.

Did you know that each next tetris piece is not generated independently from the others? The sequence of tetris pieces is generated according to the "7-bag" algorithm described as follows. Begin with an empty sequence, then repeat this infinitely many times:

- Among all possible permutations of the seven tetrominos, select one of them uniformly at random (this is a "7-bag").
- Then, append these seven tetrominos *in that random order* as the next seven terms of the sequence.

This ensures that there will never be "droughts" or "floods" of a particular piece, along with many other desirable mathematical properties.

For example, the first sixteen tetrominos in a Tetris game might look like this. For illustration, tetrominos belonging to the same 7-bag are shown grouped together.



Bob is writing some Tetris fanfiction, and wants to include the following scene:

- *Tetra started a game of Tetris and played for some unknown amount of time. She paused the game. Then, when she resumed, we observed that the pieces she got were t_1, then t_2, then t_3, and so on, until t_n.*

Bob cares about his story being accurate to the game mechanics of real life Tetris. Is it possible for the sequence $t_1, t_2, t_3, \ldots, t_n$ to appear consecutively somewhere in some sequence generated by the 7-bag algorithm? And if not, what is the fewest number of terms that need to be changed in order to make it so?

Input Format

Input consists of a single line containing a string of n uppercase English letters, each one of `IJLOSTZ`, where the ith letter in the string identifies the tetromino t_i.

Output Format

Output a single integer, the fewest number of terms that need to be changed in order for Bob's sequence to have the desired property. Note that if it already does have the desired property, you should output 0.

Constraints

Your program will only be tested on data that satisfies the following constraints.

Constraints
$1 \leq n \leq 100$ Each letter is one of `IJLOSTZ`

There is no need to validate in your program that these constraints hold.

Sample I/O

Input	Output
`TOIJSZLTT`	`0`

Input	Output
`ZZZZ`	`2`

Input	Output
`TILLJOLTLILTLOSTZITI`	`4`

Explanation

In the first sample input, the sequence already has the desired property. The first `T` was the last tetromino of the previous bag. Then, the next bag had the tetrominos in the order `OIJSZLT`. Finally, the last `T` was the first tetromino of the next bag.

In the second sample input, we can make the sequence have the desired property by, for example, changing the last two letters to `I` and `O`. Then, it would be possible for the first `Z` to be from one bag, and the next `ZIO` to be from another bag.

In the third sample input, we could, for example: change the fourth letter from `L` to `O`; change the seventh letter from `L` to `Z`; change the ninth letter from `L` to `S`; and change the twelfth letter from `T` to `J`.

Problem H
The Parlor

Herbert S. Sinclair is developing a suite of puzzles to challenge his grand-nephew Simon Jones, who will be visiting his manor within the coming months. Each room in his manor has a distinct gimmick, and the style of puzzle to be placed in The Parlor is as follows.

There are three boxes of three different colors: blue, white, and black, laid out in a row in that order. Each box has a statement written on it.

Image taken from PC Gamer

Each statement in the room is of one of the following forms:

- `The gems <'are' | 'are not'> in the <'blue' | 'white' | 'black'> box.`
- `The statement on the <'blue' | 'white' | 'black'> box is <'true' | 'false'>.`
- `The statement on the box with the gems is <'true' | 'false'>.`
- `The statements on the empty boxes are both <'true' | 'false'>.`
- `Exactly one box has a statement that is <'true' | 'false'>.`
- `Exactly two boxes have a statement that is <'true' | 'false'>.`

In the room, one would find the following note, explaining the rules of the game:

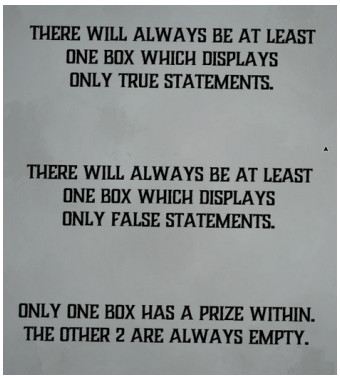


Image taken from the Blue Prince wikia

The gems, of course, are the prize. Simon must deduce which box contains the gems.

You have been hired as a playtester for the parlor puzzle. Given statements that would be placed on these boxes, determine if they have a unique solution, admit multiple solutions, or are paradoxical. Additionally, if there is a unique solution, determine which box contains the gems.

Input Format

The first line of input contains a single integer T, the number of test cases. Then, the descriptions of the T test cases follow.

Each test case is described by four lines: the statement on the blue box, then the statement on the white box, then the statement on the black box, and finally a "separator" line consisting of the `-` character repeated 75 times.

Output Format

Output the answer to each test case in its own line.

- If—according to the rules of the game—it should have been impossible for the boxes to display these statements, output `PARADOX`
- If the answer can be uniquely deduced from the statements (and the rules of the game), output the color of the box with the gems: one of `blue` or `white` or `black`
- If the given statements (and rules of the game) do not determine a unique answer, output `AMBIGUOUS` instead.

Constraints

Your program will only be tested on data that satisfies the following constraints.

Constraints
$1 \leq T \leq 400$ Each statement exactly follows one of the above formats.

There is no need to validate in your program that these constraints hold.

Sample I/O

Input

```
4
Exactly two boxes have a statement that is false.
The gems are in the white box.
The gems are in the black box.
---------------------------------------------------------------------------
The statements on the empty boxes are both true.
The statement on the box with the gems is false.
Exactly one box has a statement that is true.
---------------------------------------------------------------------------
The gems are in the blue box.
The gems are in the white box.
The gems are in the black box.
---------------------------------------------------------------------------
The statement on the white box is true.
The statement on the black box is true.
The statement on the blue box is true.
---------------------------------------------------------------------------
```

Output

```
blue
black
AMBIGUOUS
PARADOX
```

Explanation

Let's imagine how one might solve the first test case by hand.

- Suppose the statement on the blue box is false.
  - By the rules of the game, there are either one or two false statements.
  - If it is not the case that there are two false statements, then there must be *only one* false statement.
  - But we know where that false statement is—it's on the blue box!
  - Therefore, the statements on the white and black boxes must both be true.
  - But hold on, that's a **contradiction**. By the rules of the game, there should only be one box with the gems.
  - We therefore conclude that the statement on the blue box **cannot be false**.
- We have concluded that the statement on the blue box is true.
  - Therefore, there *are* exactly two false statements.
  - Since the statement on the blue box is known to be true, we can conclude that those two false statements are the ones on the white and black boxes.
  - If the gems are in neither the white box nor the black box, then only one possibility remains—the gems are in the `blue` box!

Let's imagine how one might solve the second test case by hand.

- Suppose the statement on the black box is true.
  - If there is only one true statement, and we assumed it to be on this box, then that means the other two boxes must have false statements.
  - That tells us this information: there is a false statement among those on the empty boxes; and the statement on the box with the gems is **true**.
  - Therefore, the gems are in the black box, the only box with a true statement (and also, it is indeed the case that both empty boxes have false statements).
- Suppose the statement on the black box is false.
  - By the rules of the game, there are either one or two true statements.
  - If it is not the case that there is one true statement, then there must be *two* true statements.
  - Therefore, the other two boxes must have true statements.
  - Therefore, the gems are in the black box, the only box with a false statement (and also, it is indeed the case that both empty boxes have true statements).
- So it turns out that we are actually not sure about which statements are true and which are false. But despite that, it *is* a logical surety that the gems are in the `black` box.

Let's think about the third test case.

- The gems could be in any of the boxes!
- Whatever the case, there would be one true statement and two false statements, which is consistent with the game rules.
- Since it is impossible to tell definitively which box contains the gems, we answer that it is `AMBIGUOUS`

Let's think about the fourth test case.

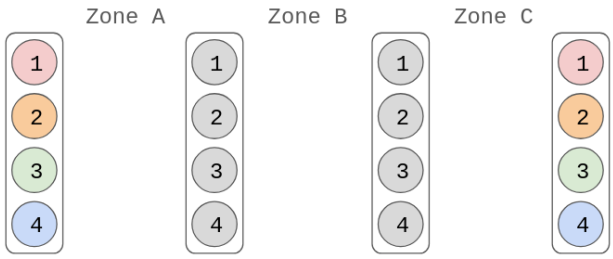
- Logically, we see that it must be the case that either all statements are true, or all statements are false.
- But, the game rules said that there should always be at least one true statement and at least one false statement.
- Since logic and the game rules are not consistent, the answer is `PARADOX`

Problem I

The Rainbow Connection

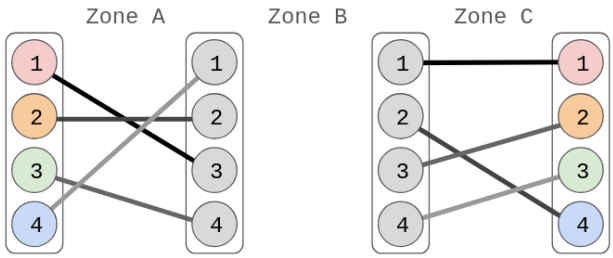
Alice, Bob, and Cindy are playing a co-op puzzle game called *The Rainbow Connection*.

In one of the puzzles, there are four columns arranged in a row, each with n nodes labeled 1 to n from top to bottom. The spaces between the columns are labeled Zone A, Zone B, and Zone C from left to right. Here is an illustration of the setup with $n = 4$.



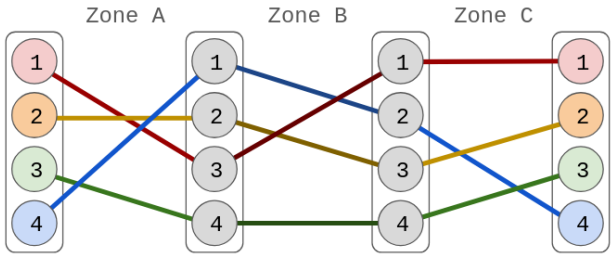
In this game, you place wires in each of the zones, with each wire connecting a node on the left with some node on the right.

Alice and Cindy got a bit excited and immediately started drawing wires in Zones A and C respectively. Alice drew n wires, but made it so that there is no node connected to two wires; so did Cindy. For example, with $n = 4$, they might have done the following.



Only after they did this did they read the rules. They must draw wires such that, for each i from 1 to n: current emitted from the leftmost node i will—if it follows the wires they've drawn—terminate on the rightmost node i.

Oops. Oh well, Bob can still save them, right? In the example from earlier, the task can still be completed if Bob draws his wires as follows.



Given the manner in which Alice and Cindy drew their wires, determine how Bob must draw his wires in order for the task to be completed. It can be shown that the task is always possible.

Input Format

The first line of input contains a single integer n.

The second line of input contains n space-separated integers $a_1, a_2, \ldots, a_n$, each a *distinct* integer from 1 to n. This means that Alice connected node i on her left with node a_i on her right.

The third line of input contains n space-separated integers $c_1, c_2, \ldots, c_n$, each a *distinct* integer from 1 to n. This means that Cindy connected node i on her left with node c_i on her right.

Output Format

Output n space-separated integers $b_1, b_2, \ldots, b_n$, each a *distinct* integer from 1 to n. This means that Bob will connect node i on his left with node b_i on his right.

If there are multiple possible answers, any will be accepted.

Constraints

Your program will only be tested on data that satisfies the following constraints.

Constraints
$2 \leq n \leq 100$ $a_1, a_2, \ldots, a_n$ are distinct integers from 1 to n. $c_1, c_2, \ldots, c_n$ are distinct integers from 1 to n.

There is no need to validate in your program that these constraints hold.

Sample I/O

Input	Output
4 3 2 4 1 1 4 2 3	2 3 1 4

Input	Output
7 1 2 3 4 5 6 7 1 2 3 4 5 6 7	1 2 3 4 5 6 7